专 题 报 告

基于可微 RankIC 损失函数的深度学习选股策略——机器学习选股系列研究之一

方 正 证 券 研 究 所 证 券 研 究 报 告

分析师

曹春晓 登记编号：S1220522030005

联系人 郭樾

相 关 研 究

《指数成分调整将影响哪些股票？2025 年 12 月指数成分股调整冲击测算》2025.11.30

《储能需求爆发驱动行业景气上行，一键投资储能电池全产业链标的——国证新能源电池指数投资价值分析》2025.11.11

《2025 年 12 月沪深 300、中证 500 等指数成分调整预测》2025.11.09

《FOF 基金美股配置市值及平均仓位双双增长，部分产品关注港股红利策略指数基金》2025.11.07

《多智能体投资交易系统 TradingAgents 部署与应用——AI Agent 赋能金融投研应用系列之一》2025.09.07

《个股日内成交量分布特征与日内流动性弹性刻画——多因子选股系列研究之二十三》2025.08.27

深度学习是机器学习的一个分支，它通过模拟人脑神经网络的复杂结构，让机器能从海量数据中自动学习和提取高层次的规律特征。金融时间序列，尤其是股票市场数据，以其高噪声、非平稳、非线性的复杂特性而著称，这恰好对传统量化模型的预测能力构成了严峻挑战。深度学习凭借深层神经网络对高噪声、非平稳、非线性金融时序数据的强大拟合能力，实现了从海量原始数据中进行端到端的特征提取与模式识别，为超越传统策略的绩效边界提供了可能。

传统量化将 Rank IC 作为因子的主要评价指标。在量化选股中，最终的决策是横截面上的排序，而非对个股收益的精确预测。传统的 MSE 或交叉熵损失是代理损失，它们优化了一个与最终排序目标并不完全一致的中介目标。直接使用 Rank IC（即 Spearman 相关系数）作为损失函数，遵循了决策一致性原则，使模型的优化方向与我们的投资目标直接对齐，从而避免了代理损失可能带来的目标偏离。然而，在 TensorFlow 框架中，构建损失函数时，Spearman 相关系数由于其中排序函数不可导，实现起来较为困难。

为实现以 Rank IC 为损失函数的深度学习训练，本文提出了一种直接以 Spearman 相关系数为损失函数配合按交易日切割 mini-batch 的端到端训练范式。第一章提出了以交易日为单位的批数据切割方法，介绍了 TensorFlow 框架下的迭代逻辑以及后续会用到的关键组件，包括 TensorFlow 训练循环逻辑以及代码、小批量梯度下降、大规模数据集专用存储格式 TFRecord、以交易日切割股票批数据集的合理性、以及数据洗牌 Reshuffle。第二章介绍了基于 Sigmoid 的软排序（Sigmoid Ranking）与神经排序（Neural Sort）两种可微排序法，作为构建 Rank IC 损失函数的关键组件。

基于以上逻辑基础与理论基础，本文第三章提出了一个用于优化线性组合的深度网络结构，基于方正金工团队前期研究所构建的特色高频量价因子进行因子聚合。训练结果在 20 日收益率的测度下取得了 12.48%的 Rank IC 与 5.41 的 Rank ICIR，验证了本方法在提升现有模型性能方面的实用价值。

风险提示：

本报告基于历史数据分析，历史规律未来可能存在失效的风险；市场可能发生超预期变化；各驱动因子受环境影响可能存在阶段性失效的风险。

正文目录

1 模型训练的理论基础与技术基础 4
1.1 训练循环逻辑 5
1.2 小批量梯度下降 7
1.3 大规模数据集存储格式 TFRecord 7
1.4 以交易日切割股票批数据集的合理性 9
1.5 数据洗牌 Reshuffle 9
2 构建可微 Rank IC 作为深度学习训练的损失函数 9
2.1 构建 Pearson 相关系数的损失函数 10
2.2 基于可微排序构建 RankIC 损失函数 10
2.2.1 基于 Sigmoid 的软排序 10
2.2.2 神经排序 Neural Sort 11
2.2.3 基于 TensorFlow 的损失函数构建 13
3 以深度网络进行线性模型的优化 13
3.1 参数设定情况 13
3.2 因子聚合效果 15
4 总结与展望 21
5 风险提示 22

图表目录

图表 1：tf.keras.Model.train_step 源码 5
图表 2：自定义训练循环代码样例 6
图表 3：训练循环示意图 6
图表 4：各类梯度下降法描述 7
图表 5：编码序列化样例代码 8
图表 6：解码序列化样例代码 8
图表 7：Pearson 相关系数损失函数 10
图表 8：硬排序的表示形式 10
图表 9：指示函数的 Sigmoid 近似逼近 11
图表 10：Sigmoid 软排序的表示形式 11
图表 11：Sigmoid 软排序的代码实现 11
图表 12：硬排序矩阵定义 12
图表 13：硬排序矩阵变式 12
图表 14：软排序矩阵的行公式 12
图表 15：软排序矩阵的收敛性 12
图表 16：Neural Sort 软排序的代码实现 13
图表 17：Spearman 相关系数的代码实现 13
图表 18：训练数据集划分示意图 14
图表 19：网络结构示意图 14
图表 20：不同学习率下训练集损失迭代情况 15
图表 21：训练结果 RankIC 与 IR 概览 16
图表 22：20 日收益率测度下累计 RankIC 16
图表 23：多空组合净值走势对比 16
图表 24：alpha 分层测试表现统计 17
图表 25：等权对照组分层测试表现统计 17
图表 26：alpha 十分组及多空净值走势 17
图表 27：等权对照组十分组及多空净值走势 18
图表 28：alpha 分年度表现 18
图表 29：等权对照组分年度表现 18
图表 30：alpha 分选股域表现 18
图表 31：等权对照组分选股域表现 18
图表 32：网络分解示意图 19
图表 33：两侧网络 RankIC 与 IR 概览 19
图表 34：两侧网络累计 RankIC 19
图表 35：两侧网络多空组合净值走势 19
图表 36：累积滚动训练数据集划分示意图 20
图表 37：累积滚动训练 RankIC 与 IR 概览 20
图表 38：累积滚动训练累计 RankIC 21
图表 39：累积滚动训练多空组合净值走势 21

1 模型训练的理论基础与技术基础

近年来，以深度学习为代表的人工智能技术正以前所未有的力量推动着各个领域的范式变革。作为机器学习的一个重要分支，深度学习通过构建具有多个隐藏层的神经网络模型，模拟人脑的层次化认知机制，从而具备了对海量原始数据进行端到端特征学习的卓越能力。与依赖人工特征工程的传统机器学习方法不同，深度学习能够自动从数据中挖掘出从低级到高级的复杂、非线性规律，在计算机视觉、自然语言处理等领域取得了里程碑式的成就。

金融时间序列数据，尤其是股票市场数据，以其高噪声、非平稳、非线性的复杂特性而著称，这恰好对传统量化模型的预测能力构成了严峻挑战。而深度学习的强大表征学习与动态模式捕捉能力，为解决这一难题提供了崭新的视角和有力的工具。诸如循环神经网络（RNN/LSTM/GRU） 对时序依赖关系的有效建模、卷积神经网络（CNN） 对局部形态特征的敏锐提取，以及 Transformer 架构对长程全局关联的精准捕捉，都使得直接从历史价格、行情快照乃至宏观新闻、社交媒体舆情等多模态数据中，预测股票价格的未来走势成为可能。

传统的深度学习选股模型通常将预测问题构建为回归（如预测收益率）或分类（如预测涨跌方向）任务，并相应地采用均方误差（MSE）或交叉熵损失函数作为模型的优化目标。在这一范式下，模型的训练过程依赖于一个关键的超参数——batch_size，即以固定的样本数量随机切割训练数据，形成一个 mini-batch 进行梯度下降。

然而，这种以固定 batch_size 和 MSE 损失为核心的经典训练组合，在面向以“排序”为核心的选股任务时，暴露出了较为明显的局限性。在投资实践中，投资的最终收益往往不依赖于对股票绝对值的精准预测，而更依赖于股票间的相对排名顺序。一个理想的选股模型，其预测得分应与股票的未来表现（如收益率）具有较高的秩相关性，即拥有较高的 Spearman 相关系数。尽管 MSE 损失能够优化预测值的绝对误差，但它与最终的排序指标之间存在显著的“目标不一致性”——对于一个 MSE 更小的预测，其秩相关性未必更高。

这一“目标不一致”问题，自然催生了一个更为直接的思路：能否将 Spearman 相关系数本身作为损失函数，来引导模型直接学习正确的股票排序？尽管这一想法直观且诱人，但其实现面临着两大核心困难：其一是排序的不可微性，Spearman 相关系数的计算核心在于排序操作，而排序函数本身是一个离散、非平滑的运算，其梯度几乎处处为零或不存在，无法直接嵌入基于梯度下降的深度学习框架中进行端到端训练；其二是批量数据的无关性，以固定 batch_size 随机采样构成的 mini-batch，其内部包含的股票可能来自不同的交易日、不同的市场环境，在这种混杂的背景下计算出的单个 mini-batch 的排名，与我们在每个同一交易截面内所关心的股票截面排名在金融逻辑上是不完全匹配的。

为实现以 Rank IC 为损失函数的深度学习训练，我们采用了两种可微排序技术去近似逼近计算排序过程，从而将 Spearman 相关系数这一非平滑的排序指标转化为一个可微的损失函数。同时，我们采用了按交易日切割的 mini-batch 策略，确保了每个批次内进行排序的股票处于同一时间截面，更加符合金融逻辑。

本章旨在介绍后续训练需要用到的关键组件，包括从代码的角度介绍 TensorFlow 的训练逻辑、Mini-Batch 的思想及其意义、如何借助 tfrecord 处理股票训练数据、以交易日切割股票数据集的意义、以及 reshuffle 技术。

1.1 训练循环逻辑

TensorFlow 是一个由 Google Brain 团队开发并维护的、开源的端到端机器学习平台。它的名字源于其核心操作——“张量（Tensor）”在多维数据流图（Flow）中的流动与计算。它提供了一个全面、灵活的工具生态系统，从底层的数学运算到高级的模型构建，支持从研究原型到生产部署的整个机器学习工作流。Keras 最初是一个独立的高级神经网络 API，2019 年随着 TensorFlow2.0 的发布，被直接集成到 TensorFlow 中，成为其官方的高阶 API，用于构建和训练模型。 Keras 作为一个高级、用户友好的接口，它封装了底层的复杂操作，让用户能够用更简洁、更直观的代码来快速构建和实验模型。
对于某一轮训练迭代中的某批数据，其单次训练逻辑在 tf.keras.Model 类中的 train_step 方法中：

图表1:tf.keras.Model.train_step 源码

```
def train_step(self, data):
    x, y, sample_weight = data_adapter.unpack_x_y_sample_weight(data)
    # Run forward pass.
    with tf.GradientTape() as tape:
        y_pred = self(x, training=True)
        loss = self.compute_loss(x, y, y_pred, sample_weight)
    self._validate_target_and_loss(y, loss)
    # Run backwards pass.
    self.optimizer.minimize(loss, self.trainable_variables, tape=tape)
    return self.compute_metrics(x, y, y_pred, sample_weight)
```

资料来源：方正证券研究所

TensorFlow 为自动微分提供了 tf.GradientTape API，即计算某模型网络相对于某输入项的梯度。TensorFlow 会将在 tf.GradientTape 上下文内执行的相关运算记录到“条带（Tape）”上。TensorFlow 随后会使用该条带通过反向模式计算记录的模型网络的梯度，并通过优化器 optimizer 进行模型的权重更新，以此实现反向传播。
如果需要自定义训练算法，又想使用 fit 函数的便捷功能（例如回调、内置分布支持或步骤融合），可重写上述单步训练函数 train_step。在主训练函数 fit 中，会针对每批次数据调用该函数。
在大多数情况下，fit 函数功能已经非常完备，但在极少数情况下，fit 可能不够灵活而无法满足某些训练的需要，此时需要自定义训练循环。对于主训练函数 fit，其代码逻辑可以简写为以下样例：

图表2:自定义训练循环代码样例

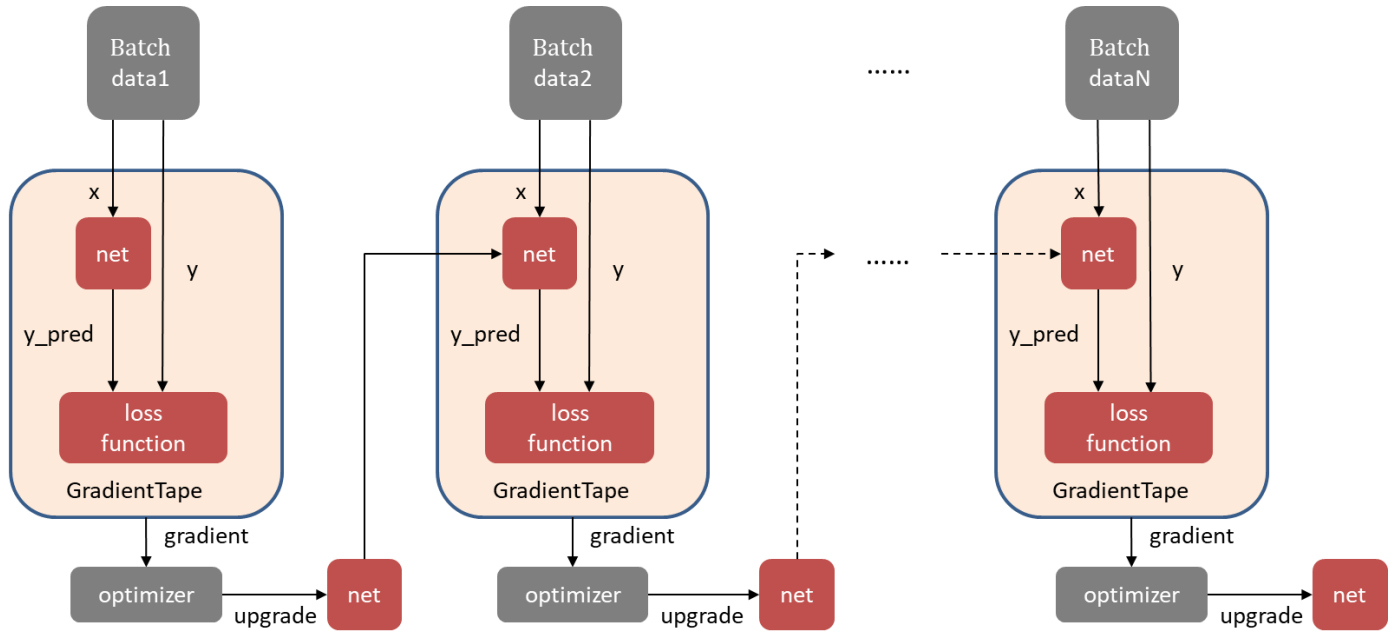
```python
import tensorflow as tf
from tqdm import tqdm


def do_training(dataset: tf.keras.datasets,
                net: tf.keras.Model,
                loss_fct: tf.keras.losses.Loss,
                optimizer: tf.keras.optimizers.Optimizer,
                metric: tf.keras.metrics.Metric,
                epochs):
    train_loss_tracker = tf.keras.metrics.Mean(name="loss")
    for epoch in range(1, epochs + 1):  # for loop epoch
        print("正进行训练 Epoch {}/{} :".format(epoch, epochs))
        train_loss_tracker.reset_state()
        metric.reset_state()
        for x, y in tqdm(dataset):
            with tf.GradientTape() as tape:  # 记录运算
                y_predict = net(x, training=True)  # training=True使得模型进入推理状态
                loss = loss_fct(y, y_predict)  # 计算loss
            gradients = tape.gradient(loss, net.trainable_variables)  # 计算梯度
            optimizer.apply_gradients(zip(gradients, net.trainable_variables))  # 执行梯度
            train_loss_tracker.update_state(loss)  # 更新记录loss数据
            metric.update_state(y, y_predict)  # 更新记录metric数据
```

资料来源：方正证券研究所

1) 在上述训练函数中，我们创建了两个嵌套循环，一个用于轮次，一个用于轮次内的批数据处理。
2) 在每个轮次开始之前，对 loss 计算器和 metric 计算器作初始化。
3) 从训练集中抽取一个批数据后，在 tf.GradientTape 代码块中对该数据进行预测并计算损失。
4) 通过 tape 针对每个可训练变量计算针对损失的梯度，然后用优化器来执行梯度下降步骤，并更新模型权重以及 loss 和 metric 计算器数据。

图表3:训练循环示意图



资料来源：方正证券研究所

1.2 小批量梯度下降

在深度学习中，Batch（批次） 指的是在模型训练的一次迭代中，同时用于计算梯度并更新模型权重的一组样本（又称 mini-batch）。在 TensorFLow 框架中，tape 往往会记录一个 mini-batch 内的运算，并生成对应的一组梯度，其可被视为全数据集真实梯度的一个带噪声的无偏估计。尽管单个 mini-batch 梯度是嘈杂的，但从长期看，它的平均方向与真实梯度完全一致，这保证了小批量梯度下降的收敛性，并且，其固有的噪声更在优化过程中起到了积极的正则化作用，有助于提升模型的泛化能力（Bottou，1991）。

Batch 是深度学习训练中协调效率（速度、内存）与效果（稳定性、泛化能力）的核心机制，Batch 存在的目的就是为了让算法可以又快又准，或者说是追求速度与避免局部最优值之间的一个权衡。Batch Size 指的是每个 mini-batch 内的样本数量。选择合适的 Batch Size 是一项艺术，它需要在多个方面进行权衡，Batch Size 的大小可能对模型的性能和训练时间产生重大影响。

大 Batch 带来更高的计算效率，更稳定的计算过程，但是泛化能力不足，硬件要求高。使用大批量的情形下，算法每秒会看到更多的实例，GPU 能更好地利用其大规模并行计算能力，理论上计算效率更高。由于梯度是基于大量样本的平均值，所以方向更准确，收敛路径更平滑。但是，经验表明，使用大 Batch 训练的模型，其在新数据上的表现（即泛化能力）往往不如小 Batch 训练的模型。同样是因为算法会看到更多实例，更多的计算过程需要被记录，这导致需要更多的显存来同时存储这些样本的中间计算结果。反之，使用小 Batch 的情形下，显存占用小，泛化能力可能更好，但是梯度噪声大，损失曲线震荡剧烈，需要调整合适的学习率。

Mini-Batch 策略是一个经典的工程学折衷，完美地协调了矛盾的需求。对于 Batch Size=1 的随机梯度下降，虽然其更新极其频繁，初期收敛快，但是缺点是梯度噪声巨大，优化路径震荡剧烈，难以稳定收敛，且无法利用硬件并行计算优势。对于 Batch Size 取全体训练集的全梯度下降，虽然其梯度方向最稳定、最准确，但是缺点是计算缓慢，无法在线学习，且容易陷入尖锐的极小值（Keskar，2016）。Mini-Batch 策略它选择了一个适中的 Batch Size 大小，使得一次计算可以处理多个样本，极大地利用了现代 GPU 的并行计算能力，计算吞吐量远高于逐个样本处理；通过对一个批次的样本求平均梯度，它平滑了单个样本带来的极端噪声，使得每次更新的方向都比单样本更具代表性，收敛路径更稳定可控。

图表4:各类梯度下降法描述

梯度下降法	Batch Size取值	描述
全梯度下降(FGD)	全体训练集	使用整个数据集计算损失函数的梯度，然后更新一次权重。
随机梯度下降(SGD)	1	从数据集中随机抽取一个样本，计算梯度并立即更新权重。
随机平均梯度下降(SAGD)	1	对数据集中所有样本分别计算梯度，以所求梯度的均值更新权重。
小批量梯度下降(MGD)	(1, 全体训练集)	每次抽取一个批次的样本，计算平均梯度后更新权重。

资料来源：方正证券研究所

1.3 大规模数据集存储格式 TFRecord

为优化大规模数据集下的训练效率，本研究利用 TensorFlow 原生的 TFRecord 数据格式构建数据管道。

TFRecord 是 TensorFlow 的首选数据存储格式，它是一个为高效处理大规模数据集而设计的二进制文件格式。它将大量零散的数据样本（如图像、文本、数值

特征）转换成一个连续的、二进制的数据流，序列化后打包进一个或几个大文件中，以实现高效、连续的读取。

TFRecord 的提升效率主要体现在 I/O 性能和训练流水线效率上。其核心原理在于，将每个数据样本（如图像、标签及关联特征）封装并序列化为 tf.train.Example 协议缓冲区消息，最终将整个数据集整合为若干个连续的二进制大文件。此举在存储层面，通过合并海量小文件，将低效的随机 I/O 转化为高效的顺序 I/O，极大提升了磁盘读写吞吐量并减少了寻道开销。在读取与训练层面，TFRecord 与 tf.data API 无缝协作，支持数据的并行解码与流水线预取，使得繁琐的数据预处理过程得以与 GPU 计算并行执行，有效消除了 I/O 瓶颈，确保了训练过程中计算资源的持续饱和利用。

由于 TFRecord 与 tf.data API 协同工作所带来的数据流水线优化，使用 TFRecord 数据集会导致模型训练过程中的显存占用量显著下降。该方案将数据解码等预处理操作从 Python 端移至 TensorFlow 计算图内部，消除了系统主内存中的冗余中间副本，从源头降低了内存压力。这种主机内存的稳定与高效，进而为 GPU 显存分配器创造了稳定的工作环境，使其无需因 I/O 瓶颈和数据延迟而保留过多冗余显存。

图表5:编码序列化样例代码

```python
import tensorflow as tf


def get_serialized_object(y, x, symbols, date):
    features = tf.train.Features(
        feature={
            "data": tf.train.Feature(
                bytes_list=tf.train.BytesList(value=[tf.io.serialize_tensor(x).numpy()])),
            "label": tf.train.Feature(
                bytes_list=tf.train.BytesList(value=[tf.io.serialize_tensor(y).numpy()])),
            "stock": tf.train.Feature(
                bytes_list=tf.train.BytesList(value=[tf.io.serialize_tensor(symbols).numpy()])),
            "date": tf.train.Feature(
                bytes_list=tf.train.BytesList(value=[tf.io.serialize_tensor(date).numpy()])),
        }
    )
    example = tf.train.Example(features=features)
    return example.SerializeToString()
```

资料来源：方正证券研究所

图表6:解码序列化样例代码

```python
import tensorflow as tf


def decode_serialized_object(example, return_symbol=False):
    feature_description = {
        'data': tf.io.FixedLenFeature([], tf.string),
        'label': tf.io.FixedLenFeature([], tf.string),
        'stock': tf.io.FixedLenFeature([], tf.string),
        'date': tf.io.FixedLenFeature([], tf.string),
    }
    parsed_example = tf.io.parse_single_example(example, features=feature_description)
    x = tf.io.parse_tensor(parsed_example['data'], tf.float32)
    y = tf.io.parse_tensor(parsed_example['label'], tf.float32)
    if return_symbol:
        stock = tf.io.parse_tensor(parsed_example['stock'], tf.string)
        date = tf.io.parse_tensor(parsed_example['date'], tf.string)
        return x, y, stock, date
    else:
        return x, y
```

资料来源：方正证券研究所

1.4 以交易日切割股票批数据集的合理性

标准深度学习理论建立在数据独立同分布的假设上。而金融时间序列数据具有显著的序列相关性、非平稳性和异方差性，严格违反了独立同分布假设。因此，直接套用独立同分布数据设计的理论是不合理的。但我们希望通过构建以交易日切割批次的方式，在批次内部尽可能满足独立同分布的假设，从而为深度学习理论的应用提供一个合理的近似。

跨越长时间段的金融时间序列数据是非平稳的，然而在足够短的时间窗口内（例如，一个交易日），市场的基本结构和驱动机制是相对稳定的，其可以被近似地看作一个平稳过程（Dahlhaus,1997）。因此，在一个交易日内产生的横截面数据，可以近似地被视为来自于同一个数据分布。对于按交易日切割的 Mini-Batch，单个 Mini-Batch 内每个样本都是从当天的局部数据分布中独立抽样得到的，其内部数据可以近似满足独立同分布假设。整个训练过程于是可以理解为，模型学习一系列短期、局部平稳的数据分布，并期望模型能够捕捉到这些局部分布中存在的共性规律。

此外，从金融理论视角看，同一交易日内的股票在统一的市场微观结构（Hasbrouck,2007）和因子暴露体系（Ang,2014）下交易，这为局部独立同分布假设提供了经济学支撑。通过这种方式，我们将一个全局非平稳的优化问题，转化为一系列局部平稳的学习任务，使模型能够更稳健地捕捉市场规律。

1.5 数据洗牌 Reshuffle

Reshuffle（重新洗牌） 指的是在数据集的不同训练轮次（Epoch）之间，重新打乱数据顺序的行为。如果每个训练轮次都使用相同的数据次序，模型会学习到这种固定的数据出现模式，这不利于真正的泛化，reshuffle 确保了每个训练轮次的数据顺序都是全新的、随机的，这对于模型收敛和防止过拟合至关重要。

市场的运行并非简单的重复。如果模型按时间顺序学习，它可能会过度拟合训练期末尾的特定市场规律。当市场状态发生变化时，模型将表现不佳。

reshuffle 确保了模型在单个 Epoch 内接触到分散在不同时期的、多样化的市场环境。这种跳跃式的学习迫使模型参数更新必须兼顾各种不同的市场条件，从而找到一个在所有环境下都相对稳健的平衡点，极大地提升了模型的鲁棒性和样本外表现。

TensorFlow 将洗牌功能集成于 tf.data.Dataset.shuffle 中。其关键参数为 buffer_size，用于定义洗牌缓冲区的大小。数据集初始化时，会预先顺序加载 buffer_size 条记录到缓冲区，每次从缓冲区中随机抽取一条记录作为输出，随后从原始数据集中补充一条新记录到缓冲区，保持缓冲区大小不变，重复上述操作直到遍历完所有数据，形成一个全新的训练次序。

reshuffle_each_iteration 参数控制在不同 epoch 之间是否重新洗牌。

2 构建可微 Rank IC 作为深度学习训练的损失函数

传统因子量化将 Rank IC 作为主要评价指标。在量化选股中，最终的决策是横截面上的排序，而非对个股收益的精确预测。传统的 MSE 或交叉熵损失是代理损失，它们优化了一个与最终排序目标并不完全一致的中介目标。直接使用 Rank IC 作为损失函数，遵循了决策一致性原则，使模型的优化方向与我们的投资目标直接对齐，从而避免了代理损失可能带来的目标偏离。

2.1 构建 Pearson 相关系数的损失函数

在以交易日切割 Mini-batch 的数据集中，构建 Pearson 相关系数作为损失函数是非常简单的。在某一轮次（Epoch）内的批数据中，其输入项 x 为样本数为当日有效股票数量的多维张量，代入单神经元输出层的网络模型后，将返回与预测目标 y_true 相同形状的形如【当日有效股票数量*1】的二维张量 y_pred。此时，根据 Pearson 相关系数公式，使用 TensorFlow 内置函数计算即可：

图表7:Pearson 相关系数损失函数

```
def pearson_corr(y_true, y_pred):
    cov = tf.reduce_mean((y_true - tf.reduce_mean(y_true)) *
                         (y_pred - tf.reduce_mean(y_pred)))
    std_true = tf.math.reduce_std(y_true)
    std_pred = tf.math.reduce_std(y_pred)

    return -cov / (std_true * std_pred)
```

资料来源：方正证券研究所

2.2 基于可微排序构建 RankIC 损失函数

在深度学习框架中构建基于排名的损失函数（如 Spearman 相关系数）面临的根本性挑战源于排名操作本身的不可导性。排名函数本质上是将连续预测值映射为离散整数序列的阶跃函数。该函数在绝大多数区域内对输入的微小变化不敏感，只要预测值的相对顺序保持不变，梯度便恒为零；仅在预测值大小关系发生反转的孤立点上，函数输出产生突变导致导数不存在。这种几乎处处为零梯度、关键点不可导的特性彻底阻断了反向传播的路径，使得基于梯度的优化算法无法直接应用于以排名为基础的评价指标。

可微排序是一个连接离散排序操作与连续神经网络的重要桥梁，它允许模型在反向传播中通过排序操作进行梯度优化。本章节将介绍基于 Sigmoid 的软排序（Sigmoid Ranking）与神经排序（Neural Sort）两种可微排序法，作为构建 Rank IC 损失函数的关键组件。

2.2.1 基于 Sigmoid 的软排序

为了解决排序指标不可微的问题，研究者们提出了多种连续松弛方法。其中，Qin 等人（2010）提出了一个基于 sigmoid 的排序逼近框架。他们利用 sigmoid 函数来近似排序操作，从而可以直接对信息检索指标进行基于梯度的优化。文章指出，全局排序由所有两两比较结果决定，那么通过松弛每一对元素之间的比较操作，即可松弛整个排序。

对于一个待排序序列 $X=\{x_1,x_2,x_3,\ldots,x_n\}$，对于其元素$x_i$，其硬排序可以表示为所有其他排在其前面的元素的指示函数之和：

图表8:硬排序的表示形式

$$r_i = 1 + \sum_{j \neq i} I(x_j > x_i)$$

资料来源：方正证券研究所

其中 $I(x_j > x_i)$ 为指示函数，当 $x_j > x_i$ 时为 1，否则为 0，这个指示函数就是不可微的根源。文章将指示函数用一个平滑的、可微的概率表达来近似——元

素 j 排在元素 i 前面的概率。这个概率可以通过元素 j 和 i 差值 $x_j - x_i$ 的 Sigmoid 函数来计算。

图表9:指示函数的 Sigmoid 近似逼近

$$I(x_j > x_i) \approx g_{ji} = \frac{1}{1 + e^{-(x_j - x_i)/\tau}}$$

资料来源：方正证券研究所

其中τ为缩放因子，并满足$\tau > 0$。当$\tau \to 0$时，上述近似函数无限趋近于原始的指示函数。由此，我们可以得到x_i的平滑软排序：

图表10:Sigmoid 软排序的表示形式

$$\tilde{r}_i = 1 + \sum_{j \neq i} \frac{1}{1 + e^{-(x_j - x_i)/\tau}}$$

资料来源：方正证券研究所

Sigmoid 函数进行的是“软评估”，相比于指示函数进行的“硬判断”，它不仅仅判断谁大谁小，更量化了“确信度”。 使用 Sigmoid 函数在概念上等价于承认：“我们并不完全确信基于当前得分的排序”，我们可以将 g_{ji} 解释为，在模型存在一定预测方差的情况下，x_j 的排名实际上高于 x_i 的概率。这一概率化视角让模型变得更加稳健和合理，它不再武断地对待所有边界案例，而是允许排序存在一定的“模糊地带”，并通过优化来逐步减少这种不确定性。

在 TensorFlow 中，可以以下述代码实现 Sigmoid 软排序：

图表11:Sigmoid 软排序的代码实现

```
def sigmoid_rank(vector: tf.Tensor, temperature=1.0):
    pairwise_diff = vector - tf.transpose(vector)
    pairwise_diff = tf.nn.sigmoid(pairwise_diff / temperature)
    ranks = tf.reduce_sum(pairwise_diff, axis=1, keepdims=True)
    return ranks
```

资料来源：方正证券研究所

2.2.2 神经排序 Neural Sort

尽管 Qin 等人（2010）的开创性工作为直接优化信息检索指标提供了可行的路径，但其方法本质上是通过构造代理函数来近似最终的排序指标。这种“近似指标”的范式虽然有效，但在计算效率和通用性上仍面临挑战。它通常需要复杂的 pairwise 计算，并且为不同指标设计特定的松弛策略也非易事。

沿着“使排序可微”这一研究方向，后续工作开始探索更根本、更通用的解决方案。其中，Grover 等人（2019）提出的 Neural Sort 代表了一种范式的转变。它不再间接地近似排序指标，而是选择直接对排序操作本身进行连续松弛，从而为实现可微排序提供了一个更为优雅和强大的基础算子。

一个 n 维排序$\mathrm{z} = [z_1, z_2, \ldots, z_n]^T$是一个由唯一索引$\{1,2,\ldots,\mathrm{n}\}$组成的列表，每一个排序 z 都会与一个排序矩阵$P_z \in \{0,1\}^{n \times n}$相关联（其本质上是 z 的独热编码），其元素定义如下：

图表12:硬排序矩阵定义

$$P_z[\mathrm{i,j}] = \begin{cases} 1 & if \ \ j = z_i \\ 0 & otherwise \end{cases}$$

资料来源：方正证券研究所

令 s= $[s_1, s_2, \dots, s_n]^T$是一个长度为 n 的实值向量，令A_s表示向量 s 各元素间差值的绝对值矩阵，即$A_s[i,j] = |s_i - s_j|$，那么排序矩阵$P_{sort(s)}$可以表示为：

图表13:硬排序矩阵变式

$$P_{sort(s)}[\mathrm{i,j}] = \begin{cases} 1 & if \ \ j = argmax[(n+1-2i)s - A_s 1] \\ 0 & otherwise \end{cases}$$

资料来源：方正证券研究所

其中1表示所有元素为 1 的单位列向量。详细证明过程可参考论文的附录 B。
然而，argmax 算子是不可微的，文章提出用 softmax 算子替代 argmax 算子，从而得到一个连续松弛的排序矩阵$\hat{P}_{sort(s)}(\tau)$，其第 i 行由以下公式给出：

图表14:软排序矩阵的行公式

$$\hat{P}_{sort(s)}[i,:](\tau) = softmax[\frac{(n+1-2i)s - A_s 1}{\tau}]$$

资料来源：方正证券研究所

其中$\tau > 0$是一个温度参数，用于控制松弛程度，当$\tau \to 0$时，$\tilde{P}$收敛于真实的硬排列矩阵，当τ较大时，则得到一个更软的概率分布。松弛后的排序矩阵对于 s 中的元素是处处连续且可微的，并且拥有以下两个性质：

（1） 单峰性：对于$\forall \tau > 0$，$\hat{P}_{sort(s)}$是一个单峰行随机矩阵。进一步的，令 u 表示对$\hat{P}_{sort(s)}$逐行应用 argmax 算子的结果，有$u = sort(x)$。

（2） 收敛性：如果我们假设 s 的各分量是独立地从一个连续型分布中抽取的，则下列收敛性几乎必然成立：

图表15:软排序矩阵的收敛性

$$\lim_{\tau \to 0^+} \hat{P}_{sort(s)}[i,:](\tau) = P_{sort(s)}[\mathrm{i},:] \quad \forall i \in \{1,2,\dots,n\}$$

资料来源：方正证券研究所

Neural Sort 的核心意义在于它系统性地解决了将离散排序操作融入端到端梯度学习框架的难题。其贡献是多层次的：它并非一个临界的启发式方法，而是基于严格的数学恒等式，提供了一个具有明确数学性质的松弛；它松弛的是排列矩阵本身，这是一个元表示，一旦获得了这个松弛的排列矩阵，它可以应用于任何依赖排序的下游计算，使其成为一个通用的、与任务无关的基础构件。
在 TensorFlow 中，可以以下述代码实现 Neural Sort 软排序：

图表16:Neural Sort 软排序的代码实现

```
def neural_rank(s: tf.Tensor, temperature=1.0):
    n = s.shape[0]
    A_s = tf.abs(s - tf.transpose(s))
    scaling = tf.cast(n + 1 - 2 * (tf.range(n) + 1), dtype=tf.float32)
    P_max = tf.transpose(tf.matmul(s, tf.expand_dims(scaling, 0)) - tf.reduce_sum(A_s, axis=-1, keepdims=True))
    P_hat = tf.nn.softmax(P_max / temperature)
    ranks = tf.transpose(P_hat) @ tf.expand_dims(n - 1 - tf.range(n, dtype=tf.float32), -1)
    return ranks
```

资料来源：方正证券研究所

2.2.3 基于 TensorFlow 的损失函数构建

基于上述理论与代码，在 TensorFlow 中，Spearman 相关系数损失函数可以通过以下代码来构建：

图表17:Spearman 相关系数的代码实现

```
def spearman_corr(y_true, y_pred):
    temperature = 0.1
    rank_true = sigmoid_rank((y_true - tf.reduce_mean(y_true)) / tf.math.reduce_std(y_true), temperature)
    rank_pred = sigmoid_rank((y_pred - tf.reduce_mean(y_pred)) / tf.math.reduce_std(y_pred), temperature)

    return pearson_corr(rank_true, rank_pred)
```

资料来源：方正证券研究所

3 以深度网络进行线性模型的优化

3.1 参数设定情况

（1） 因子选取与预处理：我们的待聚合因子池为方正金工团队前期研究所构建的特色量价因子，包括“适度冒险”、“完整潮汐”、“勇攀高峰”、“球队硬币”、“云开雾散”、“飞蛾扑火”、“草木皆兵”、“水中行舟”、“花隐林间”、“待著而救”、“多空博弈”、“协同效应”、“一视同仁”、“暗流涌动”、“激流勇进”等因子及其原始因子与构成因子，总共 47 个因子，对每个因子剔除异常值、上市时间不足一年的数据、剔除 ST/PT/*ST 后，进行截面正态化处理。

（2） 预测目标：20 日收益率。我们将股票 20 日收益率剔除异常值、上市时间不足一年的数据以及 ST/PT/*ST 后，顺序进行 winsor 化（阈值参数 1%）和截面正态化。

（3） 输入项切割：对于每个有效的预测目标，回溯该时点的股票过去 20 个交易日的数据，将其上述（1）中的因子拼接成 20*47 的数据图片，并对 nan 值进行 0 填充。将每个交易日的每对数据（数据图片+预测目标）储存为一个 mini-batch。

（4） 网络结构的选择：以优化等权组合为目的，基于残差网络的思想，我们构建了一个专门用于优化既有线性模型的残差网络架构（如图表 19 所示）。左侧为一个三层堆叠的 LSTM 时序网络架构，接收整个 20*47 的数据切片，各层神经元数分别为 30/20/10，最后接一个单神经元全连接层（Dense）作为输出，激活函数采用 tanh；右侧为一个单神经元全连接层，激活函数同样采用 tanh，但只接收 15 个最新一期降频因子（即等权组合的输入项），并根据因子方向赋值初值 1 或-1，用于表示等权组合的初始权重；最后将上述两个网络块输出值 zscore 处理后进行加权合成，线性网络权重以 sigmoid(α)表示，时序网络权重以 1-

sigmoid(α)表示，其中参数α的初值为2.2，即初始状态下等权组合贡献度约为90%。

(5) 损失函数的选择：基于前文论述的sigmoid软排序法，构建Spearman相关系数为损失函数，参数τ选择0.1。对于一个正态分布，当$\tau = 0.1$时已经是一个比较好的硬排序的逼近。

(6) 数据洗牌：设定buffer_size为100，在轮迭代前对训练集进行洗牌。

(7) 训练轮次与早停：设置最大训练轮次为500，且当验证集损失在10轮之内都没有提升时，我们认为此时模型的泛化能力开始下降，提前退出循环。模型的最终权重由验证集的损失最低点决定。

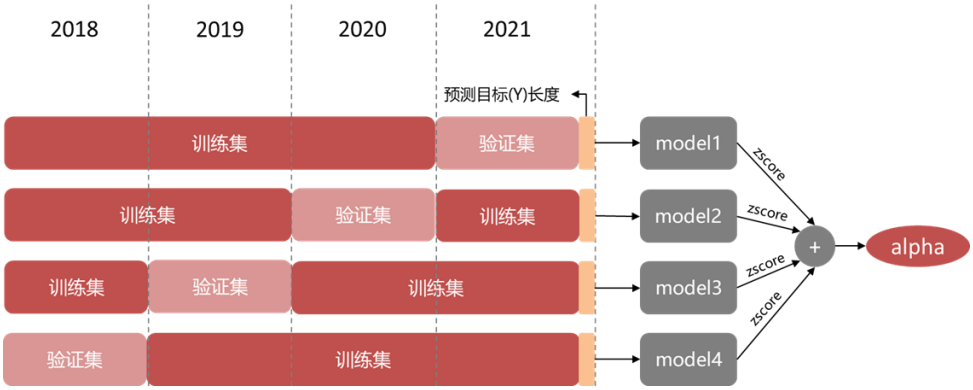
(8) 训练模式：样本内范围2018年至2021年，基于KFold交叉验证划分训练集与验证集（如图表18所示），将各子模型的结果进行zscore处理后等权合成最终的alpha。

(9) 优化器与学习率：优化器选择Adam，学习率选择0.0001。

(10) MSE对照组：以MSE为损失函数，采取batch_size=5000的原生批数据切分方式，其他参数不变，用于验证损失函数与数据切割模式的改进效果。

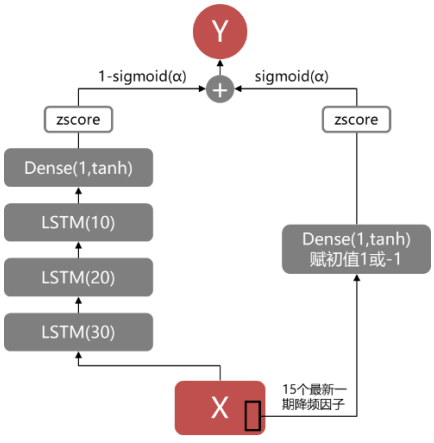
(11) 等权对照组：将（1）中提及的15个预处理后的降频因子根据RankIC的方向进行等权组合，用于对比经过深度网络训练后的alpha相对于等权组合的提升程度。

图表18:训练数据集划分示意图



资料来源：方正证券研究所

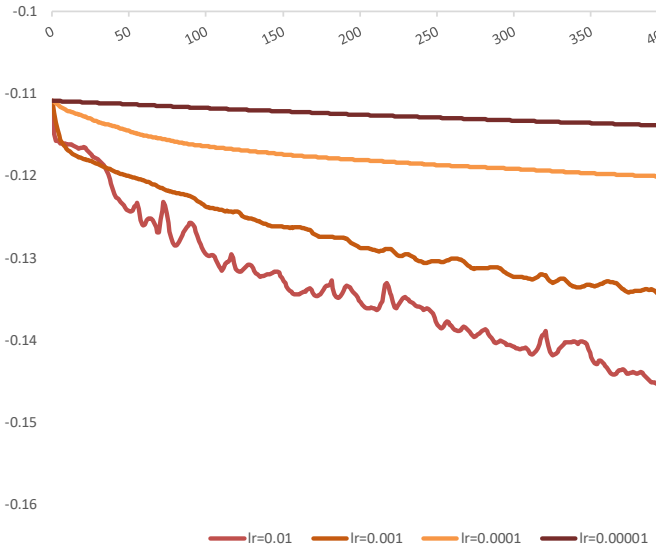
图表19:网络结构示意图



资料来源：方正证券研究所

关于学习率选择，找到一个合适的学习率非常重要，如果设置的太高，学习率可能会发散，或是围绕最优解不停震荡，如果设置的太低，训练可能会花费很长时间，并且模型可能会陷入局部最优解。为选择一个合适的学习率，我们在训练集中抽取部分样本构建一个测试数据集，以上述三层 LSTM 网络作为模型结构，以 sigmoid 软排序的 Spearman 相关系数为损失函数（参数τ选择 0.1），设定多个不同数量级的学习率，通过自定义循环观察监控每一个 mini-batch 更新模型后的总体测试数据集上的平均损失情况（如图表 20 所示），对于单个 mini-batch 来说，学习率取值 0.01 至 0.0001 皆为不错的选择，大的学习率能让算法在前期尽快达到一个比较好的结果，后期采用更小的学习率能让算法去到一个比较精细的结果，使用学习率调度的方法更佳。

图表20:不同学习率下训练集损失迭代情况



资料来源：Wind，方正证券研究所

3.2 因子聚合效果

由于数据洗牌、模型权重初始化等模块具有随机性，将上述训练重复进行多次后，将各个训练的预测输出值进行截面 zscore 后等权组合，得出最终的训练结果（下称 alpha），并统计其与对照组在样本外区间的表现（2022 年至 2025 年 10 月），并根据因子值打分作 1-10 档多空组合回测，间隔 20 交易日换仓。

在训练目标——20 日收益率的测度下，alpha 的 Rank IC 为 12.48%，Rank ICIR 为 5.41，相对于等权对照组的 Rank IC 11.46%以及 Rank ICIR 4.08 均有所提升；MSE 对照组相对于等权对照组在 Rank ICIR 较等权对照组有所提升，但是不及 alpha 的效果。

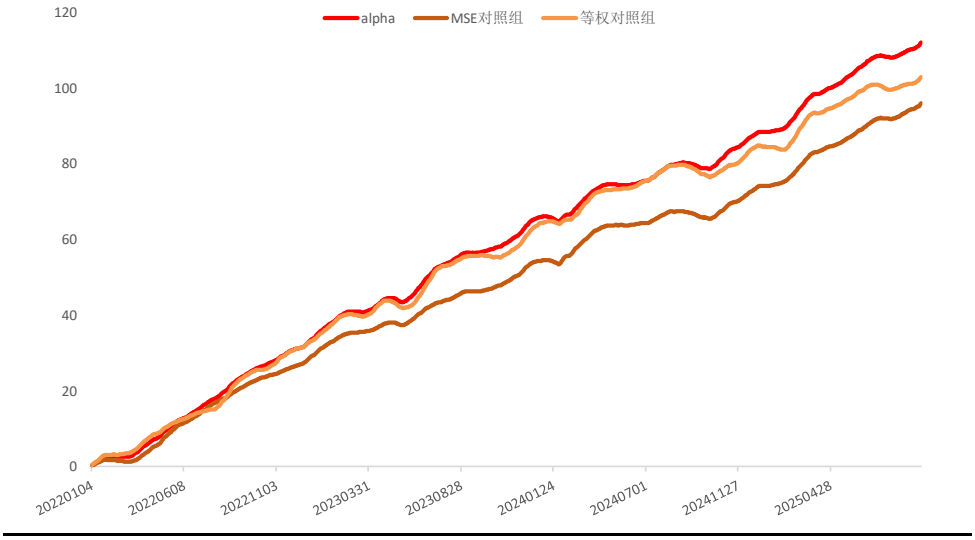
在多空组合的表现上，alpha 多空年化收益率为 42.97%，信息比率为 3.19，月度胜率为 82.61%，最大回撤为 10.90%，相对于等权组合均得到了提升。MSE 对照组虽然在 Rank IC 上不及等权对照组，但其多空年化收益率为 37.38%，信息比率为 3.00，月度胜率为 80.43%，最大回撤为 11.39%，相对于等权组合均得到了提升。

图表21:训练结果 RankIC 与 IR 概览

因子名称	Rank IC(普通)	Rank ICIR(普通)	多空年化收益率	多空年化波动率	多空信息比率	多空月度胜率	多空最大回撤
alpha	12.48%	5.41	42.97%	13.48%	3.19	82.61%	-10.90%
MSE对照组	10.69%	5.28	37.38%	12.47%	3.00	80.43%	-11.39%
等权对照组	11.46%	4.08	25.69%	14.75%	1.74	58.70%	-14.72%

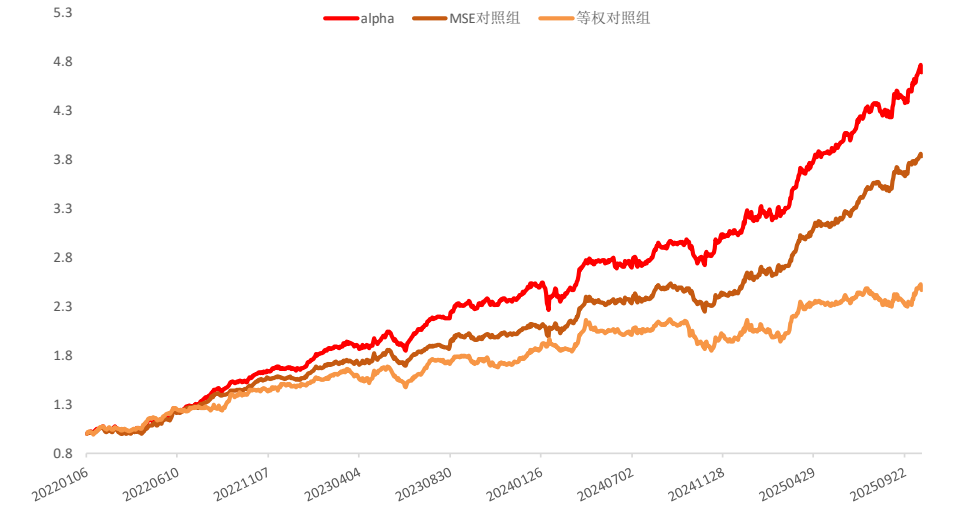
资料来源：Wind，方正证券研究所

图表22:20 日收益率测度下累计 RankIC



资料来源：Wind，方正证券研究所

图表23:多空组合净值走势对比



资料来源：Wind，方正证券研究所

从分层测试表现来看，相比于等权组合，alpha 在头部和尾部票的分离能力大大提升了，尤其是第一层和第十层的分离能力。从年度表现来看，各年度的 Rank IC 与 Rank ICIR 均强于等权组合；各年度的多空组合收益率、信息比率与月度胜率均强于等权组合。从选股域来看，alpha 在各指数内的 Rank IC 不及等权组合，算法的提升主要在于指数外的小票，这与损失函数未引入个股权重有关，后续可通过引入市值加权的 Rank IC 损失函数以提高这方面的表现。

图表24:alpha 分层测试表现统计

	年化收益率	年化波动率	信息比率	最大回撤	年化双边换手	月胜率
第1层	22.92%	12.46%	1.84	-22.99%	16.79	80.43%
第2层	16.85%	10.85%	1.55	-19.37%	19.36	69.57%
第3层	15.53%	9.93%	1.56	-17.55%	20.17	71.74%
第4层	13.14%	8.65%	1.52	-14.74%	20.43	80.43%
第5层	9.85%	7.91%	1.25	-9.33%	20.38	73.91%
第6层	7.32%	6.96%	1.05	-8.05%	19.38	67.39%
第7层	9.10%	5.84%	1.56	-5.71%	20.28	73.91%
第8层	13.96%	9.72%	1.44	-15.37%	20.50	71.74%
第9层	3.61%	9.92%	0.36	-17.25%	19.45	56.52%
第10层	-19.98%	11.88%	-1.68	-54.15%	14.85	34.78%
多空组合	42.97%	13.48%	3.19	-10.90%	32.25	82.61%

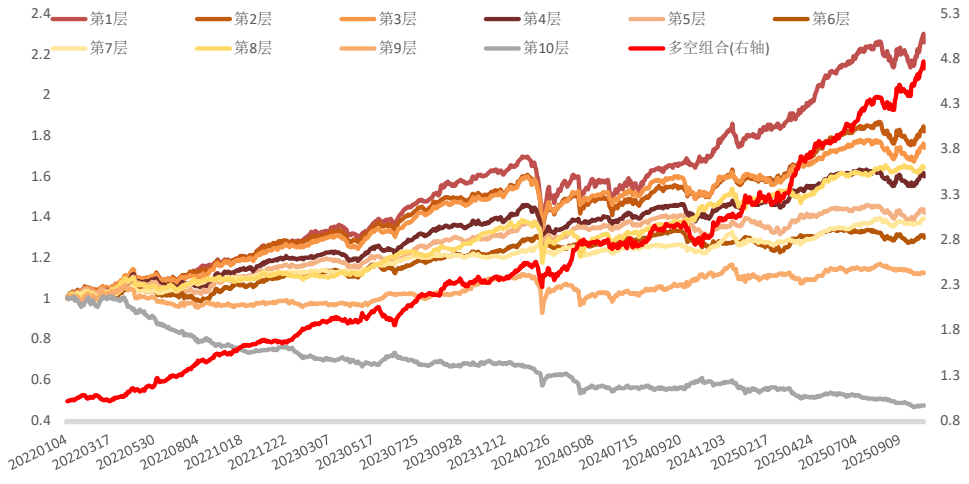
资料来源：Wind，方正证券研究所

图表25:等权对照组分层测试表现统计

	年化收益率	年化波动率	信息比率	最大回撤	年化双边换手	月胜率
第1层	19.73%	12.21%	1.62	-15.86%	17.04	69.57%
第2层	16.01%	10.55%	1.52	-16.22%	20.31	76.09%
第3层	13.79%	9.16%	1.50	-14.78%	21.25	80.43%
第4层	12.74%	7.83%	1.63	-11.24%	21.49	73.91%
第5层	11.76%	7.05%	1.67	-9.33%	21.59	76.09%
第6层	8.64%	6.18%	1.40	-6.39%	21.45	67.39%
第7层	6.52%	6.28%	1.04	-7.28%	21.51	65.22%
第8层	6.65%	7.19%	0.92	-9.43%	21.29	60.87%
第9层	6.25%	9.22%	0.68	-13.56%	20.67	63.04%
第10层	-6.15%	13.47%	-0.46	-34.22%	18.61	47.83%
多空组合	25.69%	14.75%	1.74	-14.72%	36.62	58.70%

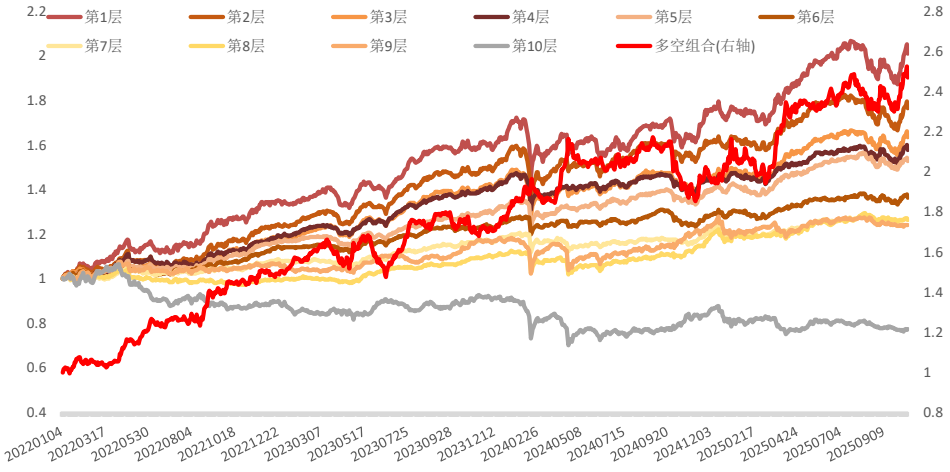
资料来源：Wind，方正证券研究所

图表26:alpha 十分组及多空净值走势



资料来源：Wind，方正证券研究所

图表27:等权对照组十分组及多空净值走势



资料来源：Wind，方正证券研究所

图表28:alpha 分年度表现

年份	Rank IC	Rank ICIR	多空组合年化收益率	多空组合年化波动率	多空组合信息比率	多空组合月度胜率	多空组合最大回撤
2022	14.08%	9.04	54.74%	11.77%	4.65	83.33%	-6.01%
2023	13.10%	5.58	39.12%	11.23%	3.48	91.67%	-9.63%
2024	9.52%	3.55	28.18%	16.34%	1.72	75.00%	-10.90%
2025	13.53%	5.65	51.62%	14.11%	3.66	80.00%	-4.33%

资料来源：Wind，方正证券研究所

图表29:等权对照组分年度表现

年份	Rank IC	Rank ICIR	多空组合年化收益率	多空组合年化波动率	多空组合信息比率	多空组合月度胜率	多空组合最大回撤
2022	13.89%	7.11	43.38%	13.64%	3.18	66.67%	-4.78%
2023	12.53%	3.96	17.22%	13.30%	1.30	50.00%	-12.30%
2024	8.67%	3.13	18.05%	15.07%	1.20	58.33%	-14.72%
2025	10.44%	3.31	23.89%	17.22%	1.39	60.00%	-10.25%

资料来源：Wind，方正证券研究所

图表30:alpha 分选股域表现

选股域	Rank IC	Rank ICIR	多空组合年化收益率	多空组合年化波动率	多空组合信息比率	多空组合月度胜率	多空组合最大回撤
沪深300成分股	4.63%	2.14	17.56%	16.45%	1.07	54.35%	-17.60%
中证500成分股	4.84%	2.23	7.74%	16.53%	0.47	54.35%	-27.77%
中证1000成分股	8.89%	3.73	28.03%	15.36%	1.83	73.91%	-13.12%

资料来源：Wind，方正证券研究所

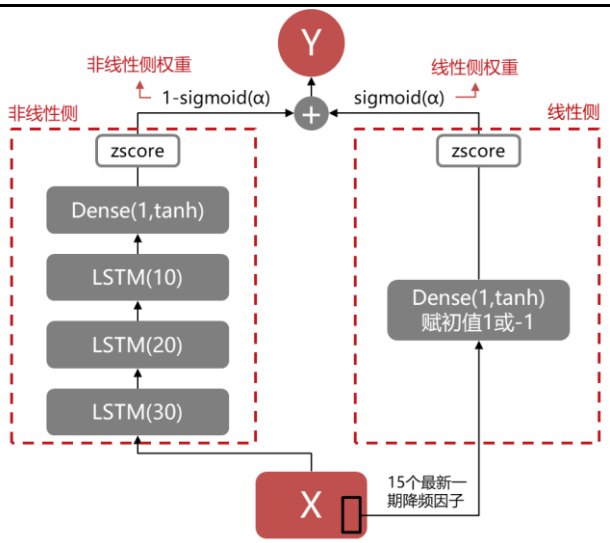
图表31:等权对照组分选股域表现

选股域	Rank IC	Rank ICIR	多空组合年化收益率	多空组合年化波动率	多空组合信息比率	多空组合月度胜率	多空组合最大回撤
沪深300成分股	7.09%	2.28	19.68%	22.36%	0.88	63.04%	-37.67%
中证500成分股	7.00%	2.35	6.43%	19.34%	0.33	56.52%	-30.98%
中证1000成分股	9.73%	3.09	20.62%	18.26%	1.13	60.87%	-22.25%

资料来源：Wind，方正证券研究所

进一步的，我们深入模型的权重数据，将网络分解为线性侧（右）和非线性侧（左）。从最终的训练结果来看，非线性侧的平均权重为 27.29%，非线性侧的平均权重为 72.71%。

图表32:网络分解示意图



资料来源：Wind，方正证券研究所

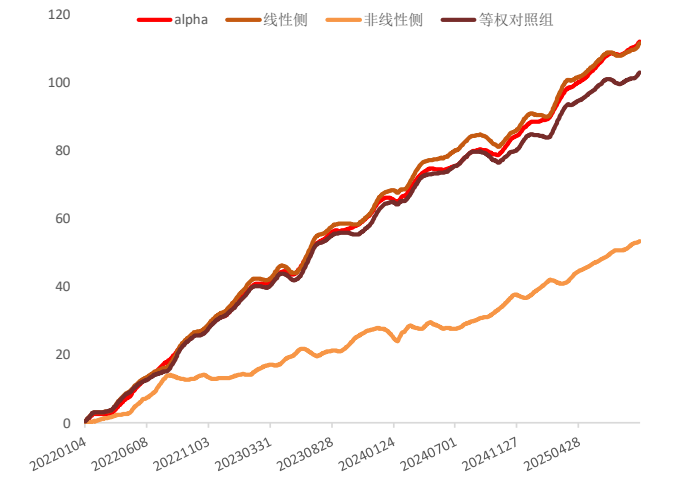
我们将两侧网络的输出结果单独作绩效统计与回测，计算方法与上文相同，同样的为了剔除随机性的干扰，输出结果取值是多个训练的平均结果。线性侧在 Rank IC 上达到了 12.42%，Rank ICIR 也是达到了 4.22，较等权对照组均有提升；非线性侧虽然在 Rank IC 和 Rank ICIR 的表现上并没有那么亮眼，但是在多空组合的表现上非常优秀，多空年化收益达到了 34.54%，信息比率达到了 3.16，月度胜率达到了 89.13%。两侧网络的结果起到了互补的作用，合并后在各个指标维度上都超越了等权对照组。

图表33:两侧网络 RankIC 与 IR 概览

因子名称	Rank IC(普通)	Rank ICIR(普通)	多空年化收益率	多空年化波动率	多空信息比率	多空月度胜率	多空最大回撤
alpha	12.48%	5.41	42.97%	13.48%	3.19	82.61%	-10.90%
线性侧	12.42%	4.22	28.62%	16.33%	1.75	65.22%	-15.73%
非线性侧	5.93%	2.73	34.54%	10.93%	3.16	89.13%	-16.57%
等权对照组	11.46%	4.08	25.69%	14.75%	1.74	58.70%	-14.72%

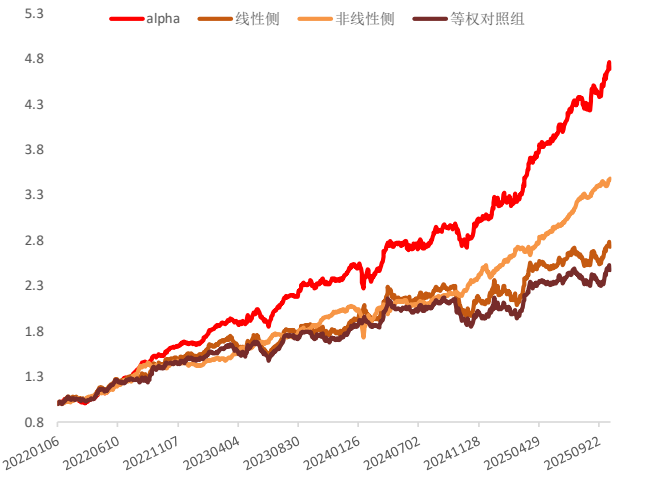
资料来源：Wind，方正证券研究所

图表34:两侧网络累计 RankIC



资料来源：Wind，方正证券研究所

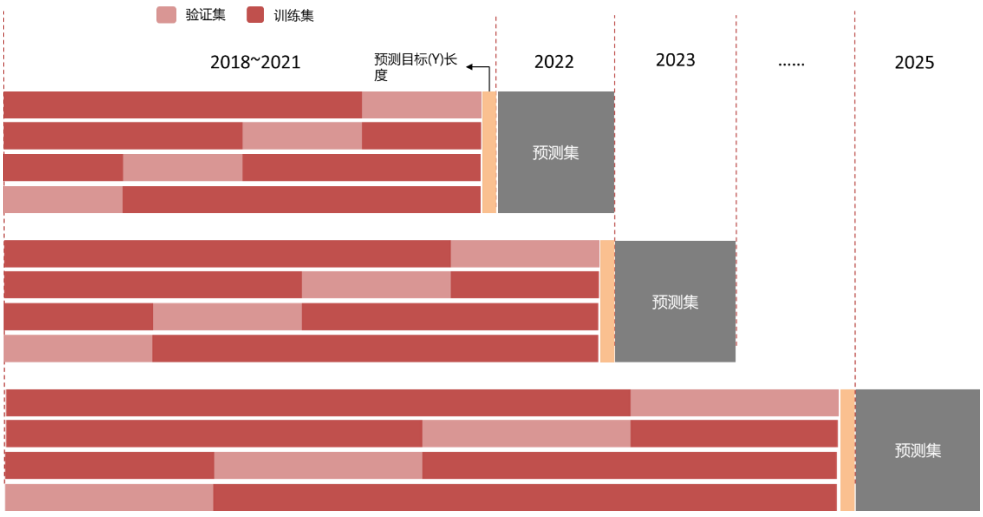
图表35:两侧网络多空组合净值走势



资料来源：Wind，方正证券研究所

进一步的，我们将上述训练模式拓展至累积数据的滚动模式。在每年的年末，我们回溯 2018 年至该年年末的数据作样本内，基于 KFold 交叉验证采用 4 等分的模式划分训练集与验证集（如图表 36 所示），将各子模型的结果进行 zscore 处理后等权合成最终的滚动预测输出，用于接下来一年的预测值。

图表36:累积滚动训练数据集划分示意图



资料来源：方正证券研究所

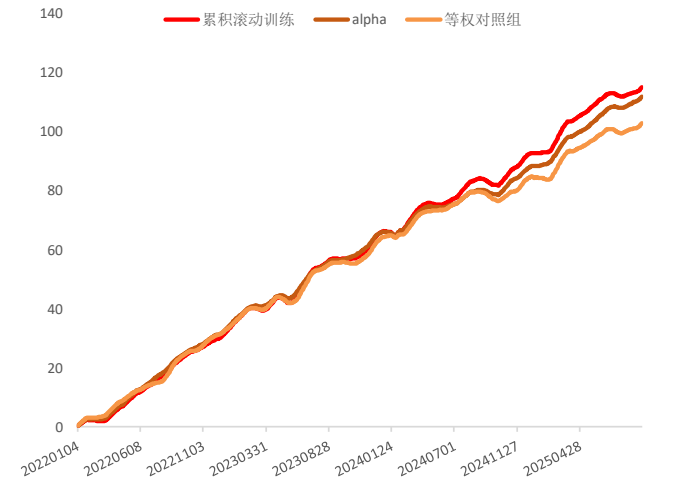
我们将预测集输出拼接后作绩效统计与回测，计算方法与上文相同。累积滚动训练在 Rank IC 维度上获得了进一步的提升，从等权对照组的 11.46%提升至 12.85%，超越了前文的固定时点的训练结果（alpha），然而其在 Rank ICIR 上稍弱，在多空组合的相关数据上也不及前文的训练结果。究其原因可能在于在后续的几个训练时点中加入了更多的小盘市场风格的数据，提升了整体解释度的同时也加剧了 Rank ICIR 上以及组合上的波动。

图表37:累积滚动训练 RankIC 与 IR 概览

因子名称	Rank IC(普通)	Rank ICIR(普通)	多空年化收益率	多空年化波动率	多空信息比率	多空月度胜率	多空最大回撤
累积滚动训练	12.85%	4.43	35.34%	16.87%	2.09	71.74%	-17.43%
alpha	12.48%	5.41	42.97%	13.48%	3.19	82.61%	-10.90%
等权对照组	11.46%	4.08	25.69%	14.75%	1.74	58.70%	-14.72%

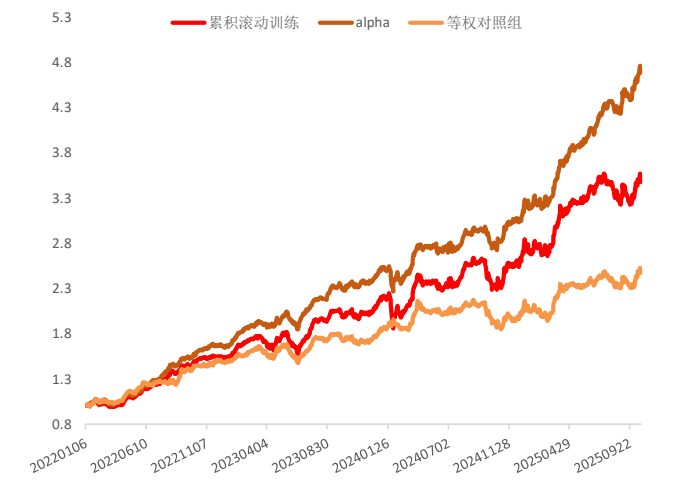
资料来源：Wind，方正证券研究所

图表38:累积滚动训练累计 RankIC



资料来源：Wind，方正证券研究所

图表39:累积滚动训练多空组合净值走势



资料来源：Wind，方正证券研究所

4 总结与展望

本研究围绕如何将 Rank IC 这一核心业绩指标直接作为损失函数融入深度学习训练流程，进行了一系列从工程基础到算法实现的探索，旨在为深度学习在量化投资中的应用提供一套可行的技术解决方案。本文的主要工作与结论可归纳为以下三点：

（1） 构建了适配金融截面数据特性的训练流程：我们在 TensorFlow 框架下，设计并实践了以交易日为单位切割 mini-batch 的关键技术。该方法确保了每个训练批次内的数据处于同一时间截面，符合计算排名指标的金融逻辑，为后续可微排序损失函数的正确计算提供了必要的数据基础。

（2） 探索并实现了可微的 Spearman 相关系数损失函数：为克服排序操作的不可微性，我们重点研究了 Sigmoid 软排序与神经排序（Neural Sort） 两种技术路径，成功将 Rank IC 转化为可用于梯度下降优化的损失函数，使模型能够直接以优化股票排名为目标进行端到端训练。

（3） 进行了将该方法应用于优化传统线性模型的实证检验：我们将所构建的损失函数，应用于一个专门用于优化即有线性模型的深度网络结构。实验结果表明，该深度网络能够有效地学习因子间的非线性组合关系，在线性模型的基础上取得了更好的因子聚合效果，训练结果在 20 日收益率的测度下取得了 12.48%的 Rank IC 与 5.41 的 Rank ICIR，验证了本方法在提升现有模型性能方面的实用价值。

尽管本研究初步验证了所提框架的可行性，但仍有诸多方面值得在未来工作中持续改进与探索：

（1） 排序算法的优化：在可微排序领域，除了 Sigmoid 软排序与 Neural Sort 之外，还有诸如 SoftRank 等强大的可微排序算法有待探索；在关键参数温度的选择上，可如部分优化器一样引入自适应调控技术，使模型在训练初期能平滑收敛，在训练后期能逼近真实排序，以兼顾稳定性与精确性。

（2） 损失函数的创新：基于可微 Rank IC 损失函数，分选股域、带权重的加权 Rank IC 损失函数有待探索，例如分市值区间计算 Rank IC 后加权、

分行业计算 Rank IC 后加权以及市值加权的 Rank IC 等；基于以交易日为单位切割 mini-batch 的关键技术与可微排序技术，除 Rank IC 以外的其他截面上的风险或收益指标（如多头加权收益率、多空加权收益率、投资组合回撤等）亦有被纳入损失函数的可能。

（3） 从“精准预测”到“最优决策”：高 IC 是高收益的必要非充分条件，优秀的 Rank IC 的预测并非最终目标，构建一个表现优异的投资组合才是核心。未来的一个关键研究方向是，探索如何将整个投资组合的构建过程纳入深度学习框架，进行端到端的组合优化训练。

5 风险提示

本报告基于历史数据分析，历史规律未来可能存在失效的风险；市场可能发生超预期变化；各驱动因子受环境影响可能存在阶段性失效的风险。

分析师声明

作者具有中国证券业协会授予的证券投资咨询执业资格，保证报告所采用的数据和信息均来自公开合规渠道，分析逻辑基于作者的职业理解，本报告清晰准确地反映了作者的研究观点，力求独立、客观和公正，结论不受任何第三方的授意或影响。研究报告对所涉及的证券或发行人的评价是分析师本人通过财务分析预测、数量化方法、或行业比较分析所得出的结论，但使用以上信息和分析方法存在局限性。特此声明。

免责声明

本研究报告由方正证券制作及在中国（香港和澳门特别行政区、台湾省除外）发布。根据《证券期货投资者适当性管理办法》，本报告内容仅供我公司适当性评级为 C3 及以上等级的投资者使用，本公司不会因接收人收到本报告而视其为本公司的当然客户。若您并非前述等级的投资者，为保证服务质量、控制风险，请勿订阅本报告中的信息，本资料难以设置访问权限，若给您造成不便，敬请谅解。
在任何情况下，本报告的内容不构成对任何人的投资建议，也没有考虑到个别客户特殊的投资目标、财务状况或需求，方正证券不对任何人因使用本报告所载任何内容所引致的任何损失负任何责任，投资者需自行承担风险。
本报告版权仅为方正证券所有，本公司对本报告保留一切法律权利。未经本公司事先书面授权，任何机构或个人不得以任何形式复制、转发或公开传播本报告的全部或部分内容，不得将报告内容作为诉讼、仲裁、传媒所引用之证明或依据，不得用于营利或用于未经允许的其它用途。如需引用、刊发或转载本报告，需注明出处且不得进行任何有悖原意的引用、删节和修改。

评级说明：

类别	评级	说明
公司评级	强烈推荐	分析师预测未来12个月内相对同期基准指数有20%以上的涨幅。
	推荐	分析师预测未来12个月内相对同期基准指数有10%以上的涨幅。
	中性	分析师预测未来12个月内相对同期基准指数在-10%和10%之间波动。
	减持	分析师预测未来12个月内相对同期基准指数有10%以上的跌幅。
行业评级	推荐	分析师预测未来12个月内行业表现强于同期基准指数。
	中性	分析师预测未来12个月内行业表现与同期基准指数持平。
	减持	分析师预测未来12个月内行业表现弱于同期基准指数。
基准指数说明		A股市场以沪深300 指数为基准；香港市场以恒生指数为基准，美股市场以标普500指数为基准。

方正证券研究所联系方式：

北京：朝阳区朝阳门南大街 10 号兆泰国际中心 A 座 17 层
上海：静安区延平路71号延平大厦2楼
深圳：福田区竹子林紫竹七道光大银行大厦31层
广州：天河区兴盛路12号楼隽峰苑2期3层方正证券
长沙：天心区湘江中路二段36号华远国际中心37层
网址：https://www.foundersc.com
E-mail：yjzx@foundersc.com